

MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Autonomous)
NEW PANVEL

PROJECT REPORT ON
RailSathi
"RAILWAY RESERVATION SYSTEM USING LINKED LIST, QUEUE AND STACK"

IN PARTIAL FULFILLMENT OF
BACHELOR OF COMPUTER SCIENCE
SEMESTER II – 2026-27

UNDER THE GUIDANCE OF
Mrs. SHUBHANGI PAWAR

SUBMITTED BY
Name: Jatin Shankar Jha
Rollno: 5693

Index

Sr. No	Contents	Page No.
1	Introduction	1
2	Objectives	1
3	Features	2
4	Source Code	3
5	Output	9
6	Conclusion	11

1. INTRODUCTION

RailSathi – Railway Reservation System is a simple Python-based application designed to manage railway passenger reservations using different data structures. The project provides basic facilities such as booking tickets, cancelling tickets, searching passengers, managing confirmed reservations, maintaining a waiting list and storing cancellation history.

The project uses a **Singly Linked List** to store confirmed passenger records. Each passenger is represented as a node containing details such as PNR number, passenger name, age, train number, source and destination. The linked list allows passengers to be inserted, searched and deleted dynamically.

A **Queue** is used to manage the waiting list. When all confirmed seats are occupied, new passengers are added to the waiting queue. The queue follows the **FIFO (First In, First Out)** principle. When a confirmed passenger cancels their ticket, the first passenger in the waiting queue is promoted to confirmed status.

A **Stack** is used to maintain the cancellation history. Cancelled passenger records are pushed onto the stack, following the **LIFO (Last In, First Out)** principle. This demonstrates the practical use of stack operations.

The project also demonstrates **Linear Search, Binary Search and Bubble Sort**. Linear Search is used to find passengers in the linked list, while Bubble Sort arranges passenger records according to their PNR numbers. Binary Search is then used to search for a passenger from the sorted records.

Thus, RailSathi provides a practical implementation of important data structures and algorithms through a railway reservation example.

2. OBJECTIVES

- To understand the practical implementation of data structures using Python.
- To develop a simple railway reservation system.
- To store confirmed passenger records using a **Singly Linked List**.
- To implement insertion and deletion operations in a linked list.
- To use a **Queue** for maintaining the waiting list.
- To understand and implement **FIFO** queue operations.
- To use a **Stack** for maintaining cancellation history.
- To implement searching techniques such as Linear Search and Binary Search.
- To implement Bubble Sort for arranging passenger records.
- To demonstrate the practical application of data structures in a real-life system.

3. FEATURES

1. Ticket Booking

The user can enter passenger details such as PNR, name, age, train number, source and destination.

2. Confirmed Reservation

The first five passengers can be stored as confirmed passengers using a Singly Linked List.

3. Waiting List

When all confirmed seats are occupied, additional passengers are added to the waiting list using a Queue.

4. Automatic Waiting List Promotion

When a confirmed passenger cancels their ticket, the first passenger in the waiting queue is automatically moved to the confirmed list.

5. Ticket Cancellation

The user can cancel a ticket by entering the passenger's PNR number.

6. Passenger Search

Passengers can be searched using their PNR number.

7. Linear Search

Linear Search is used to search passenger records stored in the linked list.

8. Bubble Sort

Passenger records can be sorted according to their PNR numbers using Bubble Sort.

9. Binary Search

Binary Search can be performed on sorted passenger records to find a passenger efficiently.

10. Cancellation History

Cancelled passenger records are stored in a Stack and can be displayed as cancellation history.

11. Menu Driven

The application provides a simple menu from which the user can select the required operation.

4. SOURCE CODE

4.1 Passenger Class

```
class Passenger:
    def __init__(self, pnr, name, age, train_no, source,
destination):
        self.pnr = pnr
        self.name = name
        self.age = age
        self.train_no = train_no
        self.source = source
        self.destination = destination
```

4.2 Singly Linked List

```
class Node:
    def __init__(self, passenger):
        self.passenger = passenger
        self.next = None
```

4.3 Queue

```
class WaitingQueue:

    def __init__(self):
        self.queue = []

    # Enqueue
    def enqueue(self, passenger):
        self.queue.append(passenger)

    # Dequeue
    def dequeue(self):

        if len(self.queue) == 0:
            return None

        return self.queue.pop(0)

    # Search waiting passenger
    def search(self, pnr):

        for passenger in self.queue:

            if passenger.pnr == pnr:
                return passenger

        return None

    # Remove passenger from waiting queue
```

```

def remove(self, pnr):

    for i in range(len(self.queue)):

        if self.queue[i].pnr == pnr:
            return self.queue.pop(i)

    return None

# Display queue
def display(self):
    if len(self.queue) == 0:
        print("\nWaiting list is empty.")
        return
    print("\n" + "=" * 75)
    print("                                WAITING LIST")
    print("=" * 75)
    position = 1
    for passenger in self.queue:
        print(
            f"{position}. "
            f"PNR: {passenger.pnr} | "
            f"Name: {passenger.name} | "
            f"Age: {passenger.age}"
        )

        position += 1

    print("=" * 75)

```

4.4 Stack

```

class CancellationStack:

    def __init__(self):
        self.stack = []

    # Push
    def push(self, passenger):
        self.stack.append(passenger)

    # Pop
    def pop(self):

        if len(self.stack) == 0:
            return None

        return self.stack.pop()

    # Display stack
    def display(self):

```

```

if len(self.stack) == 0:
    print("\nNo cancellation history.")
    return

print("\n" + "=" * 75)
print("                                CANCELLATION HISTORY")
print("=" * 75)

for passenger in reversed(self.stack):

    print(
        f"PNR: {passenger.pnr} | "
        f"Name: {passenger.name} | "
        f"Train: {passenger.train_no}"
    )

print("=" * 75)

```

4.5 Searching

```

def binary_search(passengers, pnr):

    low = 0
    high = len(passengers) - 1

    while low <= high:

        mid = (low + high) // 2

        if passengers[mid].pnr == pnr:
            return passengers[mid]

        elif passengers[mid].pnr < pnr:
            low = mid + 1

        else:
            high = mid - 1

    return None

```

4.6 Sorting

```

def bubble_sort(passengers):

    n = len(passengers)

    for i in range(n - 1):

```

```

        for j in range(n - i - 1):

            if passengers[j].pnr > passengers[j + 1].pnr:
                passengers[j], passengers[j + 1] = (
                    passengers[j + 1],
                    passengers[j]
                )
    return passengers

```

4.7 Ticket Booking and Cancellation

```

#booking
def book_ticket(confirmed_list, waiting_queue, next_pnr):
    print("\n" + "=" * 75)
    print("                                BOOK TICKET")
    print("=" * 75)
    name = input("Enter Passenger Name: ")
    age = int(input("Enter Age: "))
    train_no = input("Enter Train Number: ")
    source = input("Enter Source: ")
    destination = input("Enter Destination: ")
    passenger = Passenger(
        next_pnr,
        name,
        age,
        train_no,
        source,
        destination
    )

    confirmed_count = len(confirmed_list.to_list())

    if confirmed_count < MAX_SEATS:

        confirmed_list.insert(passenger)

        print("\nTicket booked successfully!")
        print("PNR Number:", next_pnr)
        print("Status: CONFIRMED")

    else:

        waiting_queue.enqueue(passenger)

        print("\nAll confirmed seats are occupied.")
        print("Passenger added to waiting list.")
        print("PNR Number:", next_pnr)
        print("Status: WAITING")

```

```

        return next_pnr + 1

#Cancellation
def cancel_ticket(
    confirmed_list,
    waiting_queue,
    cancellation_stack
):

    print("\n" + "=" * 75)
    print("                                CANCEL TICKET")
    print("=" * 75)
    pnr = int(input("Enter PNR Number: "))
    passenger = confirmed_list.delete(pnr)
    # Confirmed passenger found
    if passenger is not None:
        cancellation_stack.push(passenger)
        print("\nTicket cancelled successfully.")
        print("Passenger:", passenger.name)
        print("PNR:", passenger.pnr)
        # Move first waiting passenger to confirmed list
        waiting_passenger = waiting_queue.dequeue()
        if waiting_passenger is not None:
            confirmed_list.insert(waiting_passenger)
            print("\nWaiting-list passenger promoted!")
            print("Passenger:", waiting_passenger.name)
            print("PNR:", waiting_passenger.pnr)
            print("Status: CONFIRMED")

        return
    # Check waiting list
    waiting_passenger = waiting_queue.remove(pnr)
    if waiting_passenger is not None:
        cancellation_stack.push(waiting_passenger)
        print("\nWaiting-list ticket cancelled successfully.")
        print("Passenger:", waiting_passenger.name)
        print("PNR:", waiting_passenger.pnr)
        return
    print("\nPNR not found.")

```

4.8 Main Menu

```

def main():
    confirmed_list = PassengerLinkedList()
    waiting_queue = WaitingQueue()
    cancellation_stack = CancellationStack()
    next_pnr = START_PNR
    while True:
        print("\n")

```

```

print("=" * 75)
print("
                                RAILSATHI")
print("
                                RAILWAY RESERVATION SYSTEM")
print("=" * 75)

print("1. Book Ticket")
print("2. Cancel Ticket")
print("3. Search Passenger")
print("4. Display Confirmed Passengers")
print("5. Display Waiting List")
print("6. Sort Passengers")
print("7. Binary Search Passenger")
print("8. Cancellation History")
print("9. Exit")

print("=" * 75)

choice = input("Enter your choice: ")

if choice == "1":

    next_pnr = book_ticket(
        confirmed_list,
        waiting_queue,
        next_pnr
    )

elif choice == "2":

    cancel_ticket(
        confirmed_list,
        waiting_queue,
        cancellation_stack
    )
elif choice == "3":
    search_passenger(
        confirmed_list,
        waiting_queue
    )
elif choice == "4":
    confirmed_list.display()
elif choice == "5":
    waiting_queue.display()
elif choice == "6":
    display_sorted_passengers(
        confirmed_list.to_list()
    )
elif choice == "7":
    binary_search_passenger(
        confirmed_list

```

```

    )
elif choice == "8":
    cancellation_stack.display()
elif choice == "9":
    print("\nThank you for using RailSathi!")
    print("Have a safe journey! 🚂")
    break

else:

    print("\nInvalid choice. Please try again.")

```

5. Output

5.1 Main Menu

```

=====
                        RAILSATHI
                    RAILWAY RESERVATION SYSTEM
=====
1. Book Ticket
2. Cancel Ticket
3. Search Passenger
4. Display Confirmed Passengers
5. Display Waiting List
6. Sort Passengers
7. Binary Search Passenger
8. Cancellation History
9. Exit
=====

```

5.2 Successful Booking

```

=====
                        BOOK TICKET
=====
Enter Passenger Name: Jatin Jha
Enter Age: 19
Enter Train Number: 22304
Enter Source: Panvel
Enter Destination: Madgaon

Ticket booked successfully!
PNR Number: 1001
Status: CONFIRMED

```

5.3 Waiting List

```

=====
                        WAITING LIST
=====
1. PNR: 1006 | Name: Raj | Age: 19
2. PNR: 1007 | Name: Aryan | Age: 19
=====

```

5.4 Confirmed Passengers

```
=====
CONFIRMED PASSENGERS
=====
PNR: 1001 | Name: Jatin Jha | Age: 19 | Train: 22304 | Panvel -> Madgaon
PNR: 1002 | Name: Riddhi | Age: 19 | Train: 22304 | Panvel -> Madgaon
PNR: 1003 | Name: Nikhil Gaur | Age: 19 | Train: 22304 | Panvel -> Madgaon
PNR: 1004 | Name: Jaskaran | Age: 18 | Train: 22304 | Panvel -> Madgaon
PNR: 1005 | Name: Jeevan | Age: 22304 | Train: Panvel | Madgaon ->
=====
```

5.5 Ticket Cancellation & Waiting Passenger Promoted

```
=====
CANCEL TICKET
=====
Enter PNR Number: 1005

Ticket cancelled successfully.
Passenger: Jeevan
PNR: 1005

Waiting-list passenger promoted!
Passenger: Raj
PNR: 1006
Status: CONFIRMED
```

5.6 Passenger Search

```
=====
SEARCH PASSENGER
=====
Enter PNR Number: 1001

Passenger Found!
PNR      : 1001
Name     : Jatin Jha
Age      : 19
Train No. : 22304
Source   : Panvel
Destination : Madgaon
Status   : CONFIRMED
```

5.7 Sorting

```
=====
SORTED PASSENGERS
=====
PNR: 1001 | Name: Jatin Jha | Age: 19 | Train: 22304
PNR: 1002 | Name: Riddhi | Age: 19 | Train: 22304
PNR: 1003 | Name: Nikhil Gaur | Age: 19 | Train: 22304
PNR: 1004 | Name: Jaskaran | Age: 18 | Train: 22304
PNR: 1006 | Name: Raj | Age: 19 | Train: 22304
=====
```

5.8 Binary Search

```
Enter PNR Number for Binary Search: 1002
```

```
Passenger Found using Binary Search!
```

```
PNR      : 1002  
Name     : Riddhi  
Age      : 19  
Train No. : 22304  
Source   : Panvel  
Destination : Madgaon
```

5.9 Cancellation History

```
=====
                        CANCELLATION HISTORY
=====
PNR: 1003 | Name: Nikhil Gaur | Train: 22304  
PNR: 1005 | Name: Jeevan | Train: Panvel  
=====
```

6. CONCLUSION

The RailSathi – Railway Reservation System successfully demonstrates the practical implementation of important data structures and algorithms using Python. The project uses a Singly Linked List for managing confirmed passenger records, a Queue for maintaining the waiting list and a Stack for storing cancellation history.

The project also implements searching techniques such as Linear Search and Binary Search along with Bubble Sort for arranging passenger records. The automatic promotion of passengers from the waiting list after ticket cancellation demonstrates the practical application of the Queue data structure.

Overall, RailSathi provides a simple and practical example of how data structures such as Linked List, Queue and Stack can be used to solve real-world problems such as railway reservation and passenger management.